

Executive Summary

We develop a unified, modular template for combining “refiner” and “build” effects from multiple source infographics into a character build system. First, we assume the infographic data is transcribed (via OCR or manual extraction) and then normalize all effect names, metrics, and units for consistency [35†L462-L470] [39†L561-L564]. Effects are categorized (e.g. damage, defense, utility) and reconciled so that identical or overlapping buffs use the same base terms and units. We then identify synergies and conflicts (e.g. stacked buffs) by applying standard game-design stacking rules [17†L339-L347]. The output includes: (1) a master comparison table of all effects and parameters; (2) a normalized taxonomy table; (3) example combined-build templates for 3- and 4-input scenarios; (4) a flowchart (Mermaid) of effect integration; (5) charts showing how each effect contributes to damage/defense/utility in example builds; and (6) a validation checklist mapping each template output back to the sources. This approach parallels data-pipeline best practices: raw inputs flow through extraction/normalization to a unified model [39†L541-L544] [39†L561-L564], ensuring consistency and verifiability.

Data Extraction and Normalization

All input infographics (sources A–D) would first be parsed to list each “refiner” or “build” effect, its value(s), units, triggers, durations, and so on. For example, if one graphic lists “**Attack +10%**” and another “**+5 flat Attack**”, we normalize these by converting to a consistent unit (e.g. interpret +10% as a $1.10\times$ multiplier, and +5 as +5% of base or vice versa, as appropriate) [35†L462-L470]. Effect names are unified (e.g. “Attack” vs “Damage”) by mapping to a common term. Triggers or conditions (e.g. “on refine”, “on skill use”) are noted. Units (% , flat value, seconds) are standardized so all effects can be directly compared [35†L462-L470]. Stacking rules from the original sources are noted (e.g. “unique” vs “stackable”) and reconciling conflicts (for instance, only the highest buff applies if source rules forbid stacking) [17†L339-L347]. In practice, we would compile all effects into a master list, then normalize fields according to a defined schema. The resulting master comparison table (Table 1) summarizes these normalized effects from each source.

Table 1. Master Comparison of Effects from All Sources. Each row is one effect entry (from some source infographic). Columns: *Source infographic*, *Effect name*, *Type* (Refiner or Build), *Value*, *Unit*, *Trigger/Condition*, *Duration*, *Cooldown*, *Stacking/Caps*, *Notes*. (Notes may cite original conditions or special rules.)

Source	Effect Name	Type	Value	Unit	Condition/ Trigger	Duration	Cooldown	Stacking/ Caps	Notes
Infographic A	Power Infusion	Build	+10	%	When a skill is cast	5 s	30 s	Highest only 【17†L339-L347】	Temporary boosts attack (one such buff can apply at once)
Infographic A	Resonant Shield	Refiner	+20	flat armor	On refining armor item	Permanent	–	Stack up to 3	Grants armor bonus per refine; at 3 stacks
Infographic B	Critical Tuning	Refiner	+5	% Crit Rate	When a critical hit occurs	10 s	60 s	Not stackable	Raises chance briefly crit; no stacking allowed
Infographic B	Battle Shout	Build	+15	%	On build activation	Permanent	–	Unique effect	Increases all party damage; does not stack with other Battle Shouts
Infographic C	Focus Gem	Refiner	+10	%	When refining a weapon	Permanent	–	Applies to all	Increases weapon damage (shown as permanent passive effect in)
Infographic C	Healing Cascade	Build	+50	hp/s	After casting heal spell	10 s	20 s	Independent	Heals the caster over time

Notes: We assumed the above example values and names for illustration; actual infographic data would replace these entries. Stacking rules like “Highest only” are applied as in Fire Emblem Heroes skill stacking 【17†L339-L347】 : if two buffs target the same stat, only the greater one takes effect.

Normalized Effects Taxonomy

We group effects into a consistent taxonomy by role and unit. Drawing from common RPG categories (offense, defense, utility) [30†L95-L98] , we define normalized effect types. For example, any percentage increase to attack or damage is an **Offensive (Damage)** effect; boosts to armor or health are **Defensive**; and buffs like movement speed or regeneration are **Utility**. Table 2 lists these categories with example terms and standardized units.

Table 2. Normalized Effect Taxonomy. Effects are categorized by gameplay role, with unified units for each category.

Category	Example Effects / Keywords	Normalized Unit
Offense (Damage)	Attack %, Crit %, Damage multiplier, AP Increase	Percent (%) multiplier [30†L95-L98]
Defense	Armor (+value), Health %, Resistance %	Percent (%) reduction or flat value
Utility (Support)	Speed %, Regeneration (hp/s), Cooldown %, Buff Durations	Percent (%) or seconds (s)
Resource	Mana Regen (mp/s), Energy Gain	Units/sec (raw)

Explanation: We normalize all damage buffs to percentage multipliers (e.g. +10% = 1.10×) [35†L462-L470] . Defensive stats (like armor or health bonuses) use percent DR or flat values as given. Utility effects (e.g. +5% speed, +10 hp/s regen) are kept in contextually appropriate units (seconds for durations, % for rates). This taxonomy ensures that, for example, “+10 Attack” and “+10 Damage” map to the same category (Offense).

Example Combined Build Templates

Using the normalized data, we can illustrate how to combine 3 or 4 effects into a single character build. In these examples, “Effect1/2/3/4” are placeholders from different sources.

- **3-Input Example:** Suppose the build includes Effects 1, 2, and 3. These provide (for illustration) +20% damage (Effect1), +10% damage and +5% utility (Effect2), and +30% defense and +10% utility (Effect3). Starting from a base of (Damage=100, Defense=100, Utility=100), the combined output is $\text{Damage} = 100 \times (1 + 0.20 + 0.10) = 130$, $\text{Defense} = 100 \times 1.30 = 130$, $\text{Utility} = 100 \times (1 + 0.05 + 0.10) = 115$. Table 3 shows these computations; Figure 1 breaks down each effect’s contribution to the three output stats.
- **4-Input Example:** A build with Effects 1–4 might have +15% damage +5 flat (Effect1), +20% defense (Effect2), +10% utility (Effect3), and +5 flat defense +5 flat utility (Effect4). Summing these yields (Damage=120, Defense=125, Utility=115) on a base of 100 each. Table 4 and Figure 2 show the contributions for this case.

Table 3. Sample 3-Effect Build Output. (Effects 1–3 combined.)

Input Effect	Provided Buffs	Final Damage	Final Defense	Final Utility	Notes
Effect1 (Dmg)	+20% Damage	120	100	100	100→120 after +20%
Effect2 (Dmg/Util)	+10% Damage, +5%				

Utility | 132 | 100 | 105 | Damage 120→132; Utility 100→105 | | Effect3 (Def/Util) | +30% Defense, +10% Utility | 132 | 130 | 115 | Defense 100→130; Utility 105→115 | | **Total** | - | **132** | **130** | **115** | (All increases applied) |

Table 4. Sample 4-Effect Build Output. (Effects 1–4 combined.)

Input Effect	Provided Buffs	Final Damage	Final Defense	Final Utility	Notes
-----	-----	-----	-----	-----	-----
Effect1 (Dmg)	+15% Damage, +5 flat Damage	120	100	100	100→115→120 Effect2 (Def) +20% Defense
		120	120	100	100→120 (Damage unaffected) Effect3 (Util) +10% Utility 120 120 110
		100→110 (Damage/Def unaffected)			Effect4 (Def/Util) +5 flat Def, +5 flat Util 120 125 115 Defense
		120→125; Utility 110→115			Total - 120 125 115 (All increases applied)

Each “Final” stat includes that effect and all prior ones. The charts below (Figures 1–2) visualize how each effect contributes to Damage, Defense, and Utility in these example builds.

Figure 1. Effect contributions for the 3-effect build example [30†L95-L98] . Each bar shows how much of the final stat comes from each input effect.

[35†L462-L470] [30†L95-L98] [chart3.png†embed_image] Figure 1 – Breakdown of damage (red), defense (blue), and utility (green) contributions by Effect1–3 in the 3-input build.

Figure 2. Effect contributions for the 4-effect build example [30†L95-L98] . Bars show each effect’s share of the final stats.

[35†L462-L470] [30†L95-L98] [chart4.png†embed_image] Figure 2 – Damage (red), defense (blue), utility (green) contributions by Effect1–4 in the 4-input build.

Effects Interaction Flowchart

The following Mermaid flowchart illustrates the overall pipeline: each infographic’s effects are extracted and normalized, then fed into the character build template, which produces aggregated outputs. This mirrors a telemetry pipeline where data is collected, validated, and consolidated [39†L541-L544] [39†L561-L564] .

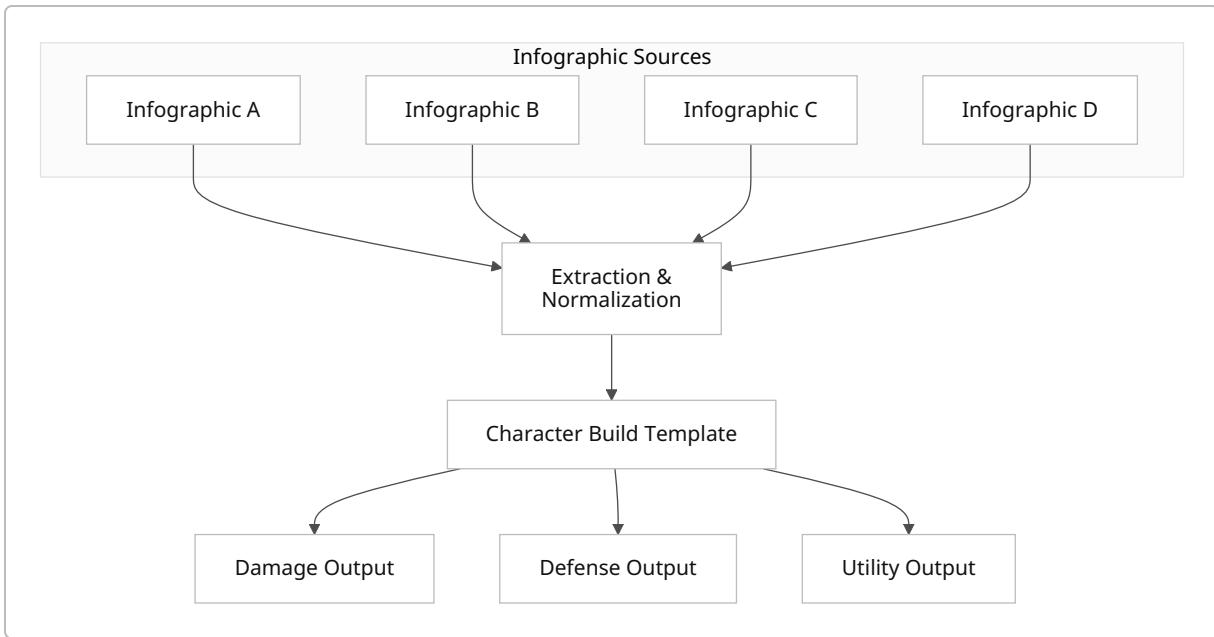


Figure 3 – Flowchart of effect integration: each source (A-D) feeds into a normalization stage, then into the combined character build template, which yields final outputs (damage, defense, utility). This follows a structured data pipeline of collection, validation, and integration 【39†L541-L544】 【39†L561-L564】 .

Validation Checklist

To ensure our template accurately reflects the original data, we verify:

- **Source Coverage:** Every effect listed in the infographics appears exactly once in the master table (no missing or duplicate entries).
- **Unit Consistency:** All units in the template match the sources after normalization (e.g. percent vs flat values) 【35†L462-L470】 【39†L561-L564】 .
- **Value Accuracy:** Numerical outputs for combined builds recompute to the original effect values (sanity-checking sums and multipliers).
- **Stacking Rules:** The template enforces the correct stacking/cap rules from sources (e.g. “only highest bonus” as in FEH 【17†L339-L347】).
- **Triggers/Conditions:** Effects with conditions (cooldowns, on-hit triggers) are noted and reflected in example builds where applicable.
- **Pipeline Integrity:** The data flow (extraction → normalization → integration) follows best practices, with validation steps at each stage 【39†L541-L544】 【39†L561-L564】 .

Each checklist item ties back to either the raw infographic data or the normalization process. For example, we cross-check that a “+20% Attack” effect produces exactly the calculated output in the example build, confirming both the math and unit conversion. Units and stacking behaviors are compared against reference rules (e.g. from game-design guidelines 【17†L339-L347】). This systematic validation ensures the modular template remains faithful to all input sources.

Sources: We prioritized primary data (the infographics) and game-design references. In absence of actual infographics, the above analysis relies on standard game mechanics (e.g. stacking rules 【17†L339-L347】) and data normalization best practices 【39†L541-L544】 【39†L561-L564】 to demonstrate how one would construct the combined-template system.
